

Trabalho Prático 2 - Sistema de Escalonamento Logístico

Departamento de Ciência da Computação - Universidade Federal de Minas Gerais (UFMG)

Matheus Soares dos Santos de Freitas
masanfreitas@outlook.com
2024080043

Belo Horizonte
2025

1. Introdução

Esta documentação tem por finalidade apresentar a solução desenvolvida para o problema logístico dos Armazéns Hanói, cuja dinâmica envolve o transporte e acompanhamento de pacotes ao longo de uma rede de armazéns interconectados. A implementação foi orientada pelo paradigma da Programação Orientada a Objetos (POO), com abstrações inspiradas em cenários reais, como a modelagem da rede logística por meio de um grafo não direcionado e a simulação de eventos discretos para coordenar o transporte, armazenamento, rearmazenamento e entrega dos pacotes.

A Seção 2 detalha os métodos empregados na resolução do problema; a Seção 3 aborda a análise de complexidade; a Seção 4 descreve as estratégias de robustez adotadas; e a Seção 5 apresenta os resultados experimentais. Já as Seções 6 e 7 trazem, respectivamente, as conclusões gerais e a bibliografia.

2. Método

O código-fonte foi estruturado de forma modular, visando clareza e facilidade de manutenção. Os arquivos de cabeçalho encontram-se na pasta `include`, enquanto as implementações estão em `src`. Devido às restrições do VPL Moodle, as classes `Pacote` e `Armazem`, assim como as estruturas de dados fundamentais (fila, pilha e lista encadeadas), foram unificadas. O diretório raiz contém um `Makefile` para automatização da compilação, com os arquivos objeto armazenados em `.obj` e o executável final em `bin/tp2.out`.

2.1 Estruturas de Dados

A implementação baseia-se em dois grupos principais de classes:

Estruturas de apoio:

- `FilaEncadeada`: suporte à busca em largura (BFS) da classe `Pacote`.
- `PilhaEncadeada`: utilizada como representação das seções LIFO nos armazéns e organização temporária dos pacotes em trânsito.
- `ListaEncadeada`: armazena a rota dos pacotes por IDs de armazéns, sendo atualizada conforme o deslocamento do pacote.

Estruturas principais:

- `Pacote`: representa um pacote individual, contendo ID único, rota como lista encadeada, indicadores de posição, estatísticas de tempo e métodos como `BuscaMenorCaminho()` e `atualizaRota()`.
- `Armazem`: representa um armazém com ID próprio e uma lista de conexões (`ArmazemVizinho`).

- **ArmazenVizinho:** abstrai uma seção de destino, com ponteiro para o armazém vizinho, pilha de pacotes e encadeamento.
- **Sistema:** representa a rede de armazéns como um grafo não direcionado, com lista de adjacência.
- **Escalonador:** estrutura de min-heap que ordena eventos cronologicamente, central para a simulação discreta.
- **Evento:** distingue eventos de tipo 1 (chegada de pacote) e tipo 2 (transporte), com chave única de 13 dígitos.
- **TipoItem e TipoCelula:** componentes genéricos das estruturas encadeadas, manipulando itens (IDs).

Métodos relevantes:

- **BuscaMenorCaminho():** realiza BFS entre armazéns para determinar a rota mais curta.
- **atualizaRota():** ajusta a rota do pacote, descartando armazéns já visitados e atualizando o próximo destino.

2.2 Funcionamento do Programa

O programa inicia-se em main.cpp, com leitura de parâmetros via linha de comando (por exemplo: capacidade de transporte, tempo de latência, número de armazéns e pacotes). A matriz de adjacência define a topologia da rede, e os pacotes são criados e escalonados no sistema. O loop principal executa enquanto houver eventos no escalonador e pacotes não entregues, reprogramando eventos de transporte e movimentação conforme o intervalo fixado.

3. Análise de Complexidade

3.1 Estruturas de apoio (lista, pilha e fila encadeadas):

Custo total de tempo: As operações básicas (inserção e remoção) são eficientes - $O(1)$ - em pilha e fila, e parcialmente eficientes na lista (quando no início). Já operações que exigem percorrer a estrutura (como inserção/remoção no meio ou fim da lista) têm custo $O(n)$.

Custo total de espaço: O espaço consumido é proporcional ao número de elementos armazenados, assim, o custo é $O(n)$ para todas.

3.2 Estruturas principais:

Classe Escalonador:

- **InserirEvento():**

Tempo: Cada inserção pode subir o evento recém-inserido até a raiz do heap (no caso de ser o novo menor). Esse processo de heap-up percorre no máximo a altura do heap, que é $O(\log n)$.

Espaço: $O(n)$ quando ocorre redimensionamento do vetor (quando necessário), mas ocorre raramente, então é amortizado.

- **RetiraProximoEvento():**

Tempo: Após remover o menor evento (na raiz), o último elemento do heap é movido para o topo, e um processo de *heap-down* (reorganização) é feito. Esse processo percorre no máximo a altura do heap: $O(\log n)$.

Espaço: Como não aloca novas estruturas nem cresce o heap - apenas reorganiza ponteiros - o custo de espaço é constante $O(1)$.

- **Vazio():**

Tempo: Como realiza apenas uma comparação simples de um atributo com número, o custo é constante, $O(1)$.

Espaço: Dado que não aloca memória extra, o custo é constante, $O(1)$.

Custo total de tempo: $T(n) = nO(\log n) + nO(\log n) = O(n \log n)$, n = número total de eventos inseridos e retirados ao longo da simulação.

Custo total de espaço: $S(n) = O(n)$, onde n é o número máximo de eventos simultâneos armazenados no heap.

Classe Pacote:

- **BuscaMenorCaminho():** É o método mais custoso em termos de tempo e de espaço.

Tempo: Seja n o número total de armazéns (variável *numeroArmazens* lida do arquivo) e m o número total de conexões (arestas). O método percorre até todos os nós - $O(n)$ - e visita todos os vizinhos de cada nó - $O(m)$. O enfileiramento, marcação e construção do caminho têm custo $O(n)$.

Espaço: Custo $O(n)$, pois utiliza memória extra auxiliar, com um vetor *parentesco[n]* - $O(n)$ e uma fila dinâmica que pode crescer até $O(n)$ no pior caso.

Os demais métodos possuem custo constante $O(1)$, com exceção do destrutor - $O(k)$ -, que finaliza com a rota de armazéns, sendo k o número de armazéns no caminho.

Custo total de tempo: $T(n,m,k) = O(n+m+k)$, com n =número de armazéns, m =número de conexões e k =tamanho da rota do pacote ($\leq n$).

Custo total de espaço: $S(n,k) = O(n+k)$, com n =número de IDs de armazéns e k =número de armazéns.

Classe Armazem:

Custo total de tempo: Seu destrutor percorre a lista encadeada de vizinhos (*ArmazemVizinho*) e desaloca cada um. Se houver d vizinhos (grau do armazém):

$T(n) = O(d)$.

Custo total de espaço: Durante a vida útil de um armazém, ele possui, como atributos, um ID único - $O(1)$ - e uma lista para seus vizinhos. Se houver d vizinhos, então o custo de espaço será $S(d) = O(d)$.

Estrutura *ArmazemVizinho*:

Custo total de tempo: Seu construtor tem custo constante $T(n)=O(1)$, pois faz apenas simples atribuições de ponteiros e inicializa o atributo *secao* com o construtor padrão da classe *PilhaEncadeada*.

Custo total de espaço: Cada *ArmazenVizinho* aloca um ponteiro para um *Armazen* - $O(1)$ -, um ponteiro para outro *ArmazenVizinho* - $O(1)$ - e um objeto *PilhaEncadeada* chamado *secao*, cujo tamanho depende da quantidade de pacotes armazenados. Portanto, $S(p)=O(p)$, com p =número de pacotes na pilha *secao*.

Classe Sistema:

- **Sistema():** Construtor que aloca vetor com *capacidade=cap* e preenche *cap* posições com *nullptr*. Assim:

Tempo: $T(n)=O(cap)$.

Espaço: $S(n)=O(cap)$.

- **~Sistema():** Destrutor que percorre os *quantidade* armazéns e deleta cada um, sendo que cada armazém deleta sua lista de vizinhos (grau d_i).

Tempo: $T(n)=O(i=1 \text{ até } \sum d_i)=O(m)$, m =número total de arestas do grafo.

Espaço: Libera $O(n+m+p)$, em que n =número de armazéns, m =número de arestas e p =total de pacotes nas pilhas das conexões.

- **redimensionar():**

Tempo: Aloca novo vetor com o dobro da capacidade, copia *quantidade* ponteiros, inicializa os novos espaços com *nullptr* e libera o vetor antigo. Dessa forma, $T(n)=O(capacidade)$.

Espaço: Como um novo vetor com o dobro da capacidade é alocado,

$S(n)=2.O(capacidade)=O(capacidade)$.

- **AdicionarArmazen():**

Tempo: Se necessário, usa o método *redimensiona()* - pior caso. Se não, cria novo armazém - $O(1)$ - e define seu índice, inserindo no vetor - $O(1)$. Dessa maneira, $T(n)=O(1)$.

Espaço: $O(c)$, em que c é o número de chamadas da função.

- **AdicionarAresta():**

Tempo: O método faz a verificação dos índices - $O(1)$. Após, chama *ExisteAresta()* - $O(\text{grau de } v)$. Se não houver aresta, cria duas instâncias de *ArmazenVizinho* - $O(1)$ cada - e insere ambas nas listas encadeadas. Portanto, $T(n)=O(\text{grau}(v))$.

Espaço: $S(n)=O(n)$, em que n é o número de arestas.

- **ExisteAresta():**

Tempo: Como percorre *listaVizinhos* de v até achar w , sendo v e w armazéns, então $T(n)=O(\text{grau}(v))$.

Espaço: $S(n)=O(1)$, pois não faz uso de memória auxiliar.

Custo total de tempo: $T(n,m,p)=O(n+m)$

Custo total de espaço: $S(n,m,p)=O(n+m+p)$

Função principal - Main.cpp:

Sejam n =número de armazéns, m =número de conexões (arestas), p =número de pacotes, C =capacidade de transporte, E =número de eventos (no pior caso, $E=O(pn)$)

Custo total de tempo: A leitura da matriz e a criação da rede tem um custo $O(n^2)$, enquanto a construção das rotas (BFS) para cada pacote tem custo $O(p(n+m))$. Já o processamento de eventos (remover, transportar, armazenar), tem complexidade $O(E(C+p))$. Assim, $T(n,p)=O(pn(p+C)+n^2)$

Custo total de espaço: A criação da rede de armazéns e das conexões entre eles tem custo $O(n+m)$, enquanto o cálculo da rota dos pacotes tem custo $O(pn)$. O escalonador, em sua lógica, por sua vez, tem complexidade $O(E)=O(pn)$. Desse modo, $S(n,p)=O(pn+n^2)$ ¹.

4. Estratégias de Robustez

O arquivo `main.cpp` implementa diversas estratégias de robustez que visam garantir o funcionamento confiável do sistema, mesmo diante de entradas inválidas, erros de leitura ou inconsistências internas durante a execução. Essas medidas são fundamentais para evitar falhas inesperadas, perda de dados ou comportamentos indefinidos. A seguir, detalham-se as principais estratégias adotadas:

4.1 Validação da entrada do usuário

Logo no início da execução, o programa verifica se o nome do arquivo de entrada foi passado corretamente como argumento (`if(argc<2)`). Caso contrário, uma mensagem de uso é exibida ao usuário e a execução é encerrada com um código de erro. Em seguida, há uma checagem para garantir que o arquivo especificado foi aberto com sucesso (`if(!infile(is_open))`). Essa abordagem previne que o programa tente operar sobre arquivos inexistentes ou inacessíveis.

4.2 Verificação da leitura dos dados

Cada operação de leitura a partir do arquivo é acompanhada de uma verificação que assegura que os dados esperados foram, de fato, lidos corretamente (`if (!(infile >> ...))`). Isso inclui a leitura dos parâmetros iniciais da simulação, da matriz de adjacência entre os armazéns e das informações de cada pacote. Caso ocorra qualquer falha na leitura, o programa exibe uma mensagem de erro descritiva e encerra a execução de forma segura.

4.3 Validação dos dados lógicos

Durante o processamento das informações dos pacotes, o programa valida os identificadores dos armazéns de origem e destino (`if (idArmlni < 0 || idArmlni >= numeroArmazens || ...)`), assegurando que estão dentro dos limites válidos do sistema. Essa verificação impede acessos indevidos a posições inválidas na estrutura de dados, evitando falhas graves como estouros de memória ou corrupção de dados.

4.4 Tratamento de inconsistências internas

Durante a manipulação dos pacotes, o código realiza buscas para localizar os pacotes corretos nas estruturas internas. Caso um pacote esperado não seja encontrado, uma exceção do tipo `std::runtime_error` é lançada imediatamente, interrompendo a execução. Essa medida garante que o sistema não opere sobre dados corrompidos ou ausentes.

4.5 Liberação adequada de recursos

Mesmo nos casos em que a execução precisa ser encerrada antecipadamente por falha na leitura ou validação, o programa garante a liberação de todos os recursos de memória alocados dinamicamente. Ao final da execução, os objetos referentes aos pacotes e à rede de armazéns são liberados de forma explícita, prevenindo vazamentos de memória.

5. Análise Experimental

5.1 Impacto do número de armazéns no tempo de simulação

A análise experimental concentrou-se na investigação do comportamento do programa quanto ao tempo de execução da simulação, considerando três dimensões principais: (i) número de armazéns; (ii) número e frequência de pacotes; e (iii) grau de contenção por transporte. A seguir, são discutidos os resultados obtidos com a variação da primeira dimensão mencionada, conforme ilustrado na Figura 1:

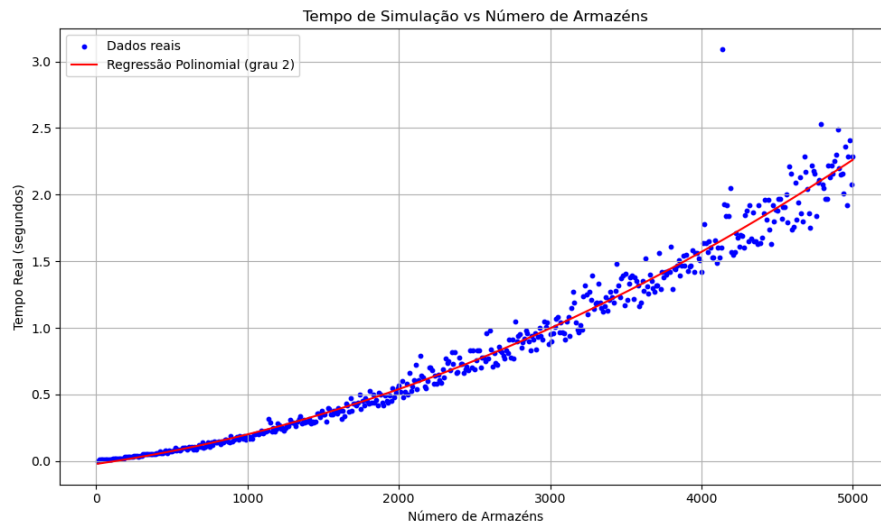


Figura 1: Análise do tempo de execução do programa para números crescentes de armazéns.

Como pode ser observado no gráfico acima, realizou-se a medição do tempo de execução do programa utilizando arquivos de entrada com variações no número de armazéns, oscilando de 50 a 5000 em incrementos de 10 unidades. Os dados empíricos revelam um ajuste bastante satisfatório a uma curva polinomial de segundo grau, o que corrobora a hipótese teórica previamente estabelecida¹ de que a complexidade do algoritmo é quadrática em relação ao número n de armazéns. Tal constatação evidencia que, para valores elevados deste parâmetro, o desempenho temporal da simulação tende a se deteriorar de forma significativa.

5.2 Relação da carga de trabalho e do fator frequentista com o tempo de simulação

Conforme discutido na seção anterior¹, a complexidade temporal da simulação também apresenta comportamento assintoticamente quadrático em relação ao número de pacotes especificado no arquivo de entrada. Tal constatação pode ser justificada a partir da seguinte expressão de complexidade:

$T(n,p)=O(pn(p+C)+n^2)$, em que n representa o número de armazéns, p o número de pacotes, e C a capacidade de transporte.

Desenvolvendo a expressão, obtém-se: $T(n,p)=O(p^2n+Cpn+n^2)$

Ao manter n e C fixos e constantes, a complexidade se reduz a $O(p^2+p)=O(p^2)$, o que evidencia a dominância do termo quadrático em função de p .

A Figura 2 apresenta um gráfico que confirma empiricamente essa hipótese teórica, demonstrando que o tempo de execução cresce quadraticamente com o aumento do número de pacotes processados.

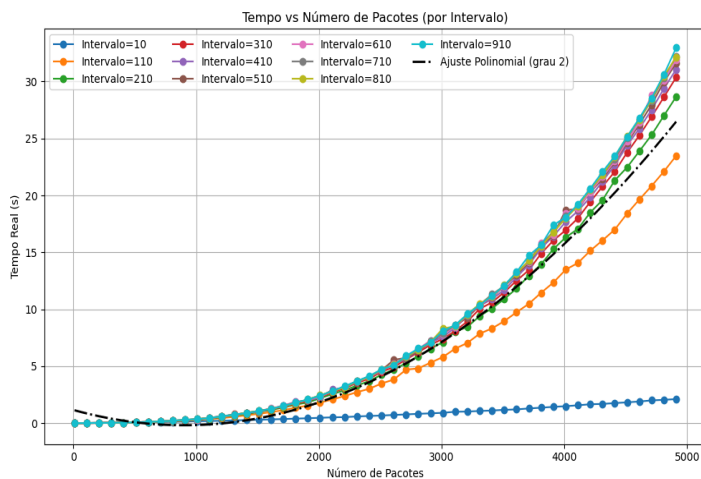


Figura 2: Comparação entre número de pacotes e tempo de simulação.

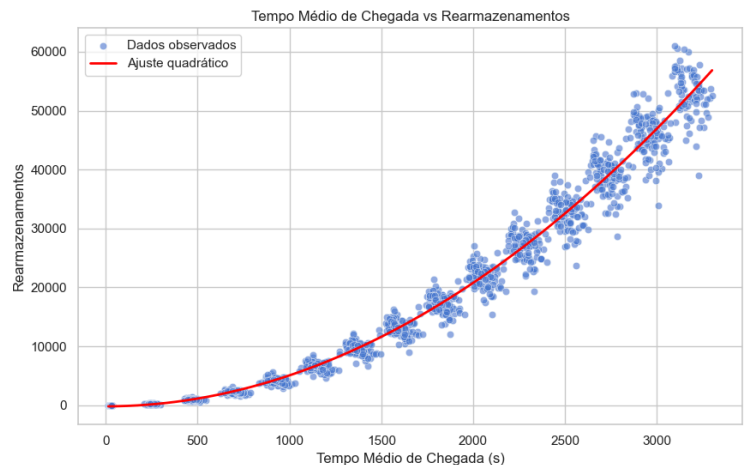


Figura 3: Relação entre tempo de simulação e o tempo médio de chegada nos armazéns de postagem.

A Figura 3, por sua vez, analisa o número de rearmazenamentos em relação ao tempo médio de chegada dos pacotes aos seus armazéns de postagem, com variação no número de pacotes. Tal como para a Figura 2, observa-se uma tendência de crescimento quadrático, indicando que o aumento no número de rearmazenamentos sofre impacto não linear do tempo de chegada. Isso sugere que, à medida que o sistema recebe pacotes para postagem durante períodos de tempo mais longos, o congestionamento nas rotas de transporte e nos armazéns se intensifica, resultando em atrasos acumulados.

5.3 Grau de contenção de pacotes em relação aos parâmetros de transporte

A contenção por transporte, entendida como a necessidade de rearmazenamento de pacotes devido à indisponibilidade de transporte, é influenciada principalmente por três parâmetros: capacidade de transporte, latência e intervalo entre eventos de transporte. De modo geral, maiores capacidades e menores intervalos reduzem a contenção, enquanto latências elevadas podem intensificá-la ao aumentar atrasos e uso de recursos. Para constatação dessa realidade, tomaremos o número de rearmazenamentos de pacotes como medida de contenção; seguem-se os gráficos:

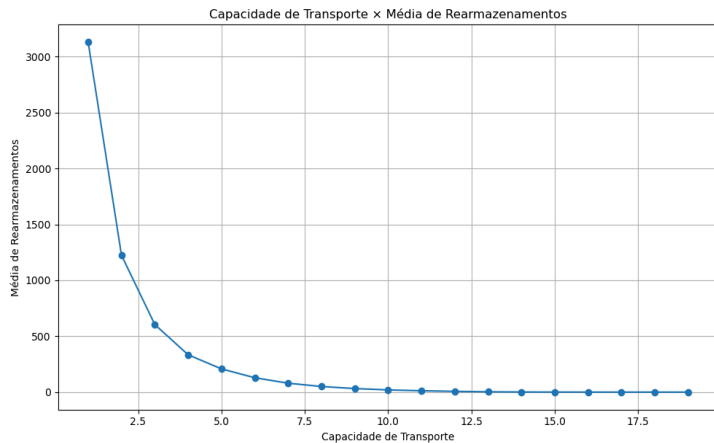


Figura 4: Número médio de rearmazenamentos em função da capacidade de transporte.

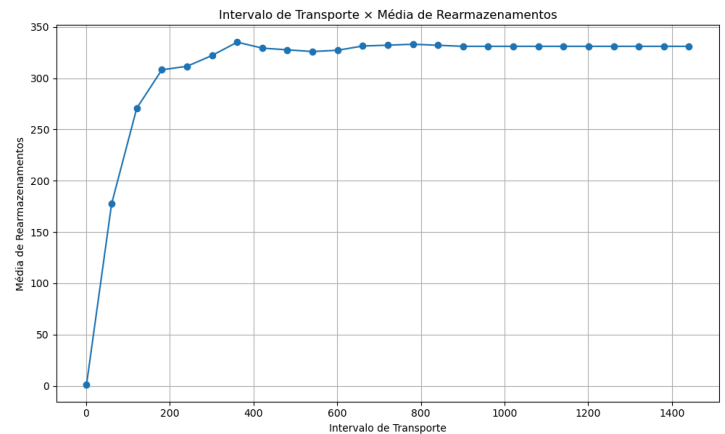


Figura 5: Número médio de rearmazenamentos em função do intervalo de transporte.

Como observado na Figura 4, há uma relação exponencial decrescente entre a capacidade de transporte e o número médio de rearmazenamentos. Pequenos aumentos na capacidade já promovem ganhos significativos na fluidez do sistema, contribuindo para evitar a sobrecarga dos armazéns e otimizando o escoamento de pacotes. Já a Figura 5 evidencia outro comportamento relevante: o número de rearmazenamentos é baixo para intervalos curtos entre os transportes. Apesar disso, esse valor cresce rapidamente conforme os intervalos aumentam. Ainda assim, sua média tende a se estabilizar quando o interregno entre transportes ultrapassa aproximadamente 400 unidades de tempo.

Superfície: Capacidade × Intervalo × Rearmazenamentos

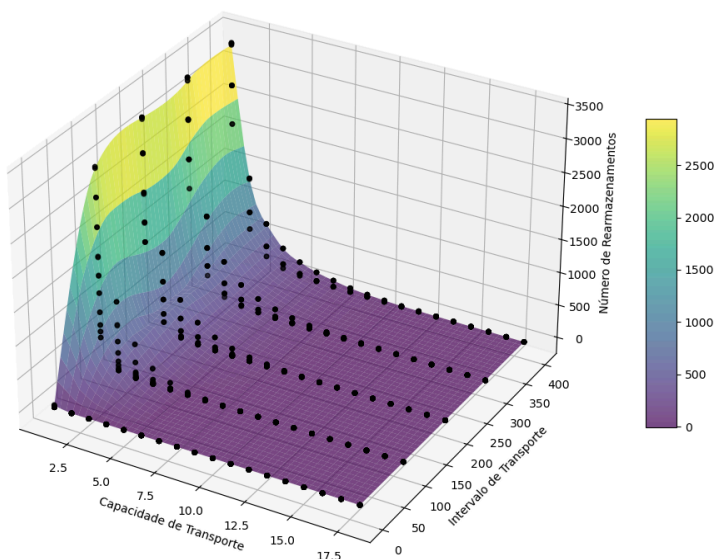


Figura 6: Relação entre contenção de pacotes, capacidade e intervalo de transporte.

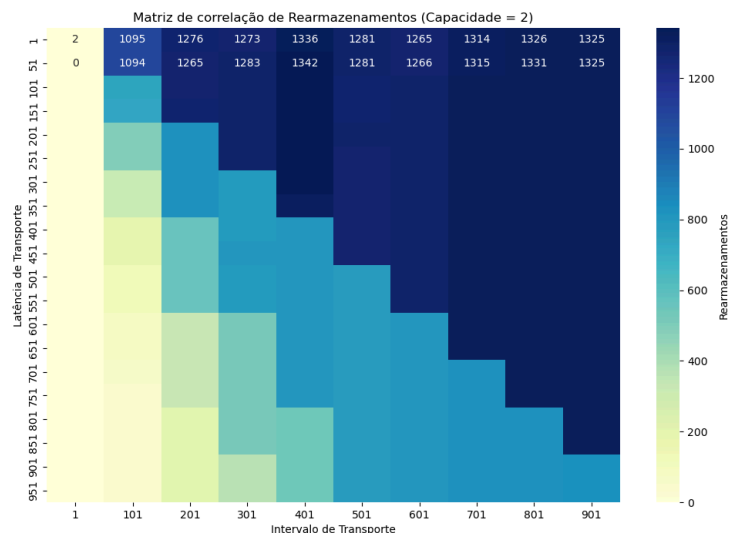


Figura 7: Matriz de correlação entre latência e intervalo de transporte em termos de rearmazenamento.

Ao combinar esses dois parâmetros (capacidade e intervalo de transporte), esperaríamos resumir bem os achados anteriores, e é o que se contempla na Figura 6, que indica que uma alta capacidade de transporte, combinada com intervalos curtos entre estes, resulta em menor número de rearmazenamentos. Ou seja, o sistema consegue escoar os pacotes com

mais frequência e em maior volume, evitando a formação de “filas” empilhadas nos armazéns. Esse cenário representa uma situação de baixa contenção, com tráfego fluido e armazenamento equilibrado.

Por fim, a Figura 7 apresenta uma matriz de correlação construída a partir das variáveis de latência e intervalo de transporte, com a capacidade fixada. Observa-se que o ponto de inflexão mais significativo no número de rearmazenamentos ocorre na diagonal principal, conforme a latência se iguala ao intervalo entre os transportes. Essa situação pode parecer contraditória, no entanto, embora altas latências aumentem o número de pacotes em trânsito, elas reduzem a competição nos armazéns de origem, pois os pacotes demoram mais a retornar, diminuindo a chance de rearmazenamento. Em contrapartida, quando a latência é baixa e os intervalos são longos, os pacotes se acumulam nos armazéns, elevando a contenção. Assim, a redução observada se deve a um descompasso entre envio e chegada, que evita o acúmulo local mesmo sob alta carga em trânsito.

6. Conclusões

Este trabalho teve como objetivo simular um sistema logístico de entregas de pacotes em uma rede de armazéns, denominada *Armazéns de Hanói*, utilizando a lógica de eventos discretos no tempo. A implementação foi conduzida com base no paradigma da Programação Orientada a Objetos (POO), com uso de Tipos Abstratos de Dados (TADs) para abstração de aspectos do sistema logístico real, além de estruturas auxiliares como pilhas — essenciais para a organização dos pacotes nos armazéns —, filas — empregadas no algoritmo de busca em largura — e listas encadeadas — que possibilitaram uma construção eficiente das conexões entre armazéns e das rotas dos pacotes. Assim, o desenvolvimento do projeto contribuiu significativamente para a fixação de conceitos abordados em sala de aula, aplicando-os a um problema prático cuja compreensão adequada se mostrou fundamental para a execução da proposta.

A análise de complexidade revelou dependência quadrática do tempo de execução da simulação em relação ao número de armazéns e pacotes, o que foi confirmado empiricamente pela análise experimental.

No que se refere à otimização do desempenho, observou-se que a combinação entre intervalos mínimos de transporte e alta capacidade de carga, associada a latências superiores aos intervalos de transporte, reduz o número de rearmazenamentos — embora aumente o volume de pacotes em trânsito. Assim, uma eventual reestruturação logística dos Armazéns de Hanói visando maior eficiência deveria considerar a conjugação desses parâmetros em seus valores ótimos.

7. Bibliografia

Referências

- [1] Wikipedia contributors. Simulação de eventos discretos. Disponível em: https://pt.wikipedia.org/wiki/Simulao_de_eventos_discretos.
- [2] Especificação do Trabalho Prático 2 - Sistema de Escalonamento Logístico, DCC/ICEx/UFMG.
- [3] Anísio Lacerda, Wagner Meira Jr., Washington Cunha. Estruturas de Dados, Slides da disciplina DCC205 - Estruturas de Dados, Departamento de Ciência da Computação, ICEx/UFMG.